SPIS TREŚCI

1. Wstęp
2. Wprowadzenie do architektury silników gier
3. Cel pracy
4. Istniejące rozwiązania

Wprowadzenie do architektury silników gier

Zrozumienie współczesnej architektury silników gier wymaga w pierwszej kolejności zdefiniowania granicy między samym silnikiem a logiką gry. Jak zauważa Jason Gregory w monografii *Game Engine Architecture* REF GEA, w praktyce rozróżnienie, gdzie kończy się silnik, a zaczyna gra bywa trudne, a granica pozostaje rozmyta. Do tego silniki gier rzadko powstają w próżni. Przykładowo silnik *Unreal Engine 1* (rok 1998) tworzony był specyficznie dla flagowej gry Epic Games *Unreal*. REF NEEDED

Historycznie, projektanci pierwszych gier nie rozróżniali pojęć “gra” i “silnik” w ogóle, gdyż nie było możliwości myślenia na takim poziomie abstrakcji. Początkowo gry nie były w ogóle “pisane” we współczesnym znaczeniu, z wykorzystywaniem języka programowania. Analiza opisu patentowego *Video Image Positioning Control System for Amusement Device* REF PONG wykazuje, iż maszyny odpowiedzialne za uruchomienie gry *PONG* (rok 1972) nie posiadały procesora, pamięci operacyjnej, a tym bardziej kodu w dzisiejszym rozumieniu. Cała “logika” gry, tj. fizyka piłki, wyświetlanie punktów, była permanentnie utrwalona na płytce TTL i jej układach cyfrowych. Mechanika gry była de facto tożsama ze sprzętem komputerowym.

Sytuacja uległa zmianie wraz z upowszechnieniem się mikroprocesorów. Poziom abstrakcji wzrósł ponad okablowanie do kodu maszynowego, jednak zasoby sprzętowe były drastycznie ograniczone, co uwidacznia poniższa tabela:

Urządzenie	Ilość pamięci RAM	Rok wydania
Oryginalny automat z PAC-MANem	2 KB	1980
Commodore 64	64 KB	1982
Atari 600XL	64 KB	1983

W konsekwencji tych ograniczeń programy, a w szczególności gry, stanowiły architektoniczny monolit. Przykładowo w grze *PAC-MAN* REF PACMAN logika gry, AI “duszków”, rysowanie grafiki i obsługa joysticka są ze sobą całkowicie wymieszane, ze względu na wymóg intensywnej optymalizacji. Każda gra była tak naprawdę swoim własnym, unikalnym silnikiem, którego nie dało się podzielić na komponenty, czy w skuteczny sposób użyć ponownie.

Już w 1965 roku Gordon Moore [REF MOORE] zaobserwował wykładniczy wzrost liczby tranzystorów w układach scalonych i przewidział, że utrzyma się on jeszcze przez wiele lat. Ów wzrost spowodował monumentalne i stabilne zwiększanie się mocy obliczeniowej komputerów/konsol i rozmiarów pamięci. Pozwoliło to na pojawienie się gier, wpierw pseudo-3D [REF P3D], np. *Wolfenstein 3D* (rok 1992) czy *DOOM* (rok 1993), a później true-3D [REF T3D], np. *Quake* (rok 1996 - PC), *Super Mario 64* (rok 1996 - Nintendo 64). Oprócz tego zwiększyły się możliwości nie tylko graficzne, ale także logiczne. Gwałtowny wzrost liczby symulowanych obiektów oraz wprowadzenie przestrzeni trójwymiarowej wywołały kryzys związany z zarządzaniem globalnym stanem gry. W grach dwuwymiarowych ten stan mógł być kontrolowany przez relatywnie prostą strukturę danych. W środowisku 3D, gdzie jednocześnie musiały współistnieć systemy detekcji kolizji, kalkulacji fizyki oraz zaawansowanej sztucznej inteligencji, utrzymanie spójności danych stało się wyzwaniem. Tradycyjne, monolityczne podejście, w którym cały program miał bezpośredni dostęp do zmiennych globalnych, prowadziło do błędów synchronizacji, trudnych do wykrycia efektów ubocznych oraz uniemożliwiało skuteczne skalowanie kodu. To drastyczne skomplikowanie systemów zarządzania stanem wymusiło ostateczną separację logiki od kodu bazowego. Ponownie programiści musieli wejść na jeszcze wyższy poziom abstrakcji. Oprócz tego, jak wspominają Bishop et al.[REF BISHOP] (1998) tworzenie tak skomplikowanych gier od zera nie byłoby ani ekonomicznie opłacalne, ani konieczne. Definiują oni silnik gry jako zbiór elementów gry, nie mających wpływu na samą zawartość gry, tj:

- pobieranie informacji z urządzeń wejścia,
- symulowanie dynamiki obiektów,
- renderowanie obrazu,
- odtwarzanie dźwięku.

O ile powyższa taksonomia odpowiada klasycznym silnikom, szczególnie gier typu FPS, TPS, MMO, MOBA, w przypadku gier strategicznych sytuacja wygląda nieco inaczej. W przypadku gier RTS lub turowych priorytety przesuwają się z symulacji ciągłej fizyki i predykcji stanów, w stronę deterministycznego zarządzania stanem wielu obiektów, np. obsługi unikalnych wymagań strategii, takich jak zarządzanie armiami, czy specyficzne algorytmy wyszukiwania ścieżek. Ponadto, wiele gier strategicznych bazuje na planszach o dyskretnym podziale na pola, np. serie *Civilization* (sześciokątne pola), *Heroes of Might and Magic* (kwadratowe i sześciokątne pola). Ten gatunkowy rozstrzał bezpośrednio ilustruje problematykę uniwersalności kodu. Problem ten definiuje Jason Gregory w monografii *Game Engine Architecture* [REF GEA], gdzie określa silnik gry jako oprogramowanie, które jest “możliwe do rozbudowania i może być użyte jako podstawa dla wielu gier bez większych modyfikacji”. Jednocześnie Gregory zaznacza, że w wielu przypadkach granica między silnikiem a grą jest rozmyta. Podkreśla on, że przesunięcie tej granicy w jedną czy drugą stronę ma bardzo jasne konsekwencje: silnik ogólnego przeznaczenia będzie bardziej uniwersalny, ale mniej przystosowany do konkretnego typu gry, zaś silnik bardziej nakierowany na dany podgatunek będzie znacznie trudniejszy lub wręcz niemożliwy do użycia w przypadku ogólnym. Ta dwojaka natura silników powoduje, że projektowanie architektury silnika to nieustanne zarządzanie kompromisem, z jednej strony elastyczność z drugiej optymalizacja pod konkretny typ rozgrywki.

Istniejące rozwiązania

Przede wszystkim warto wspomnieć, że w grach turowych nie sposób jest zbadać wszystkie możliwe ścieżki gry. Na przykładzie klasycznej gry w szachy Claude E. Shannon w pracy *Programming a Computer for Playing Chess* REF SHANNON wylicza liczbę możliwych rozgrywek na rząd wielkości 10^{120}, z założeniem średnio 40 ruchów per rozgrywka. Następnie z założeniem maszyny, która symuluje jedną rozgrywkę w czasie jednej mikrosekundy oblicza średni czas na wybranie pierwszego optymalnego ruchu na 10^{90} lat. Jest to liczba absolutnie przekraczająca możliwości obliczeniowe współczesnych systemów.

Nowoczesne strategiczne gry turowe przewyższają szachy w możliwej liczbie decyzji, ruchów i tur praktycznie zawsze. W złożonych grach typu 4X (eXplore, eXpand, eXploit, eXterminate), takich jak *Civilization VI*, standardowa rozgrywka wymaga rozegrania setek tur (średnio ok 350 REF CIV), a podczas każdej z nich gracz podejmuje wiele decyzji (średnio 10-20), każda z nich mająca wiele różnych wyników. Wskutek tego obliczenie optymalnego ruchu można uznać za niemożliwe w sensownym czasie.

Algorytmy sztucznej inteligencji bazują na rozgrywaniu tysięcy, jeśli nie milionów gier, symulując podejmowanie decyzji i ewaluując nowy stan. W związku z tym, dostarczany silnik musi spełniać paradygmat *Forward Model*, tj. musi dostarczać deterministyczną funkcję stan-przejście. Mając dany Stan (*S*) i Akcję (*A*) funkcja oblicza nowy stan (*S+1*). Ze względu na gigantyczną liczbę symulacji funkcja ta musi być możliwie zoptymalizowana pod względem złożoności obliczeniowej i pamięciowej. Oprócz tego wiele algorytmów drzewiastych (Minimax, MCTS) wymagają częstego duplikowania obecnego stanu gry na poddrzewa. Jednakże kopiowanie stanu gry często bywa bardzo kosztowne; przykładowo gra *StarCraft II* przez wiele lat nie dostarczała API typu *Forward Model* właśnie ze względu na złożoność obliczeniową i pamięciową REF STARCRAFT. Warto dodać, że optymalny *Forward Model* powinien projektować stan jako dane ciągłe w pamięci, co pozwala na klonowanie stanu za pomocą prostych operacji kopiowania bloków (cache hit) zamiast kosztownych operacji po obiektach.

Mając na celu dostarczenie szybkie i wygodne FM API dla agentów AI oraz w związku z naciskiem na gatunek TBS (Turn Based Strategy) gry *Heroes of Might and Magic*, podczas rozpatrywania istniejących rozwiązań rynkowych, prac badawczych i dedykowanych silników będę skupiał się nie na skomplikowanych potokach renderowania, synchronizacji sieciowej czy algorytmach zbierania danych od użytkownika, ale głównie na tym, w jaki sposób silniki te w deterministyczny sposób zarządzają stanem gry. Co więcej, ze względu na charakter badawczy projektu mojego silnika będę również skupiał się nie tylko na możliwości prowadzenia faktycznej rozgrywki gracz-gracz, ale także na łatwości wykorzystania silnika przez agentów, na cele treningu.

Zagadnienie dostarczenia gry z API typu FM zostało zbadane i opisane przez Diego Perez-Liebana et al. (2020) w pracy *Tribes: A New Turn-Based Strategy Games for AI Research* REF TRIBES. Jest to prosta gra turowa typu 4X, bazująca na grach typu *Civilization*, *The Battle of Polytopia*. Gra ta odbywa się na planszy 2D o rozmiarze N x N. W każdej turze gracze mogą poruszać się po planszy, atakować, rozwijać drzewo

umiejętności, etc. Powodem stworzenia tej gry był właśnie narzut związany z kopiowaniem stanu w klasycznych grach 4X (i klasycznych silnikach) na rzecz trenowania agentów AI, a celem było stworzenie lekkiego mechanizmu replikowania dla algorytmów typu SFP (ang. *Statistical Forward Planning*). Oprócz tego gra dotyka aspektu częściowej możliwości obserwowania stanu gry, tj. z perspektywy danego gracza część planszy jest ukryta za tzw. “mgłą wojny”. Środowisko gry *Tribes* z powodzeniem realizuje interfejs *Forward Model* udostępniając metody do kopiowania stanu gry, wykonywania ruchów. Aby zapewnić czystszy interfejs, *Tribes* wykorzystuje wzorzec *Command Pattern*, aby dokonać enkapsulacji i modularyzacji możliwych i wykonywanych ruchów. Choć nie zaimplementowano tego w *Tribes*, *Command Pattern* umożliwia również kolejkowanie ruchów lub implementację metody “undo”, jeżeli ruchy są deterministyczne.

Perez-Liebana et al. (2020) skupia się głównie w *Tribes* na kombinatorycznym przyroście związanym z decyzjami globalnymi, tj. co wybudować, co zbadać, i dalekosiężnym planowaniem (SFP) w warunkach niepełnej informacji.

Problemu wykładniczego przyrostu poddrzew w algorytmach drzewiastych dotyka również Santiago Ontañón w pracy *The Combinatorial Multi-Armed Bandit Problem and Its Application to Real-Time Strategy Games*[REF MULTI ARMED BANDIT]. Praca ta rozpatruje, jak sama nazwa wskazuje, problem *Combinatorial Multi-Armed Bandit*. W sytuacji gdy istnieje *N* jednostek i każda z nich ma *M* możliwych ruchów, to przestrzeń akcji w jednym kroku wynosi M^N. Ontañón skupia się na próbie oszacowania optymalnego ruchu poprzez rozważanie sposobów przeszukiwania przestrzeni i balansu eksploracja-eksploracja na podstawie autorskiego silnika gry *MicroRTS*.

Podobnie jak *Tribes*, *MicroRTS* dostarcza interfejs *Forward Model*, czyli zapewnia możliwość kopiowania stanu gry globalnym *copy* i implementuje *Command Pattern*. W przeciwieństwie do *Tribes*, *MicroRTS*, jak sama nazwa wskazuje jest grą typu RTS (Real Time Strategy), czyli czas w grze jest pseudo-ciągły (tick , a gracze podejmują decyzje w dowolnym momencie, asynchronicznie.

O ile *Tribes* optymalizuje “lekkość” samego stanu, by operacja *copy* była jak najmniej kosztowna pamięciowo i wykorzystuje płaskie struktury danych oraz prymitywy, o tyle *MicroRTS* często radzi sobie z problemem poprzez abstrakcję akcji, tj. skryptowanie zachowań jednostek, aby odciążyć Forward Model. W MicroRTS nie planuje się kroków co tick (znacznie wyższa granularność czasowa RTS), zamiast tego algorytm planuje akcje długoterminowe.

Środowiska badawcze, takie jak opisywane wcześniej *Tribes* czy *MicroRTS*, z powodzeniem realizują interfejs *Forward Model*, ich implementacja opiera się na wysokopoziomowym, obiektowym języku programowania jakim jest Java. W Javie wszystkie obiekty klas tworzone są na stercie, a zarządzanie pamięci operate jest o garbage collector. W konsekwencji obydwa mechanizmy kopiowania stanu gry są wysoce nieoptymalne, gdyż każde głębokie kopiowanie stanu gry skutkuje kaskadą braków trafienia w pamięci podręcznej (cache miss)[REF NEEDED].

Oryginalna gra *HoMM III* była napisana głównie w języku C++ [REF ICCULUS], ale kod do tej pory jest prywatną własnością (obecnie) firmy Ubisoft. *VCMI* to projekt open-source, mający na celu zbudowanie gry *Heroes of Might and Magic III* od zera w języku C++ [REF VCMI]. Silnik *VCMI* jest budowany z myślą o replikacji gry 1 do 1, z podejściem OOP. *VCMI* nie jest jednolitym programem, dziel się na proces serwera, który autoryzuje stan gry, oraz klientów odpowiedzialnych za renderowanie i obsługę gracza. Taka separacja jest dobrym rozwiązaniem do rozgrywek sieciowych, ale dla agenta AI oznaczałby duży narzut czasowy związany z tym, że komunikacja między botem a serwerem często musi przechodzić przez warstwę sieciową. Część serwerowa *VCMI* jest ściśle ustrukturyzowana i zorientowana na dostęp do obiektów przez czyste interfejsy, jednak to wprowadza problem nieciągłości danych w pamięci. Twórcy *VCMI* nie skupiali się na tym, gdyż głównym celem było utworzenie działającej i utrzymywalnej wersji gry *Heroes III*, a nie środowiska dla agentów AI. W konsekwencji tego *VCMI* nie wprowadza interfejsu *Forward Model*. // @TODO Zamiast tego *VCMI* wykorzystuje autorskie ///

Designing a PC game engine

https://www.computer.org/csdl/magazine/cg/1998/01/mcg1998010046/13rRUx0xQ1R

“Tribes” – Turn-Based Strategy Game for AI Research

https://ojs.aaai.org/index.php/AIIDE/article/view/7438

A Study and Implementation of Turn-Based Combat Systems in Role-Playing Games

https://aaltodoc.aalto.fi/items/20f87aef-461d-4358-bef4-6bc436f4131f

Developing a turn-based strategy game on unity engine

https://www.theseus.fi/handle/10024/500525

A JavaScript Finite-state Game Engine for Multiplayer Turn-based Games
https://www.theseus.fi/handle/10024/748968

Open source fan-made implementation of game Heroes of Might and Magic III

https://github.com/vcmi/vcmi

Komercyjne silniki:

1. **Firaxis** - Civilization V, Civilization VI
2. **Clausewitz Engine** - Stellaris, Crusader Kings III, Europa Universalis IV
3. **Unreal Engine** - XCOM: Enemy Unknown, XCOM 2, Gears Tactics, Triangle Strategy, Shin Megami Tensei V
4. **Unity** - Battletech, Wasteland 3, Hearthstone, Slay the Spire, Fire Emblem Engage, Pathfinder: Kingmaker, Humankind
5. **Pre-engine games**: HoMM III, Jagged Alliance

https://books.google.pl/books?id=SVvaEAAAQBAJ&lpg=PR5&ots=6XiOtnqsMz&dq=enacting%20platforms%20feminist%20technoscience&lr&hl=pl&pg=PR5#v=onepage&q&f=false

Gregory, J. (2018). *Game Engine Architecture* (3rd ed.) A K Peters/CRC Press.
gra != silnik // what is a game engine?

Nystrom, R. (2014). *Game Programming Patterns*. Genever Benning.
// command, singleton, observer, double buffer, game loop, even queue

Wolf, M. J. P. (Ed.). (2008). *The Video Game Explosion: A History from PONG to PlayStation and Beyond*. Greenwood Press.
// ping archiketktura (kabelki)

Bushnell, N. K. (1972). *VIDEO IMAGE POSITIONING CONTROL SYSTEM FOR AMUSEMENT DEVICE*. (U.S. Patent No. 3,793,483) U.S. Patent and Trademark Office. https://patents.google.com/patent/US3793483A/en
pong na kabelkach nie?

Kent, S. (2001). *The Ultimate History of Video Games*. (vol. 1). Penguin Random House LLC/Three Rivers Press
N/A tbd

Eberly, D.H. (2007). *3D Game Engine Design* (2nd ed.)
coordinate system

Atari, Inc. (1986). *PAC-MAN Source code macro assembler.* [Source code]. AtariWiki. https://atariwiki.org/wiki/Wiki.jsp?page=PAC-MAN
kod pacmana ziuuu

Braul,. A. (2024). *TetrisNESDisasm*. [Source code]. GitHub. https://github.com/CelestialAmber/TetrisNESDisasm

COMMODORE MANUAL
https://www.commodore.ca/manuals/c64_users_guide/c64-users_guide.htm

MOORE'S LAW
https://www.cs.utexas.edu/~fussell/courses/cs352h/papers/moore.pdf

GAME ENGINE DEFINITON
https://www.cs.princeton.edu/courses/archive/spring2001/cs598b/papers/bishop98.pdf

SOREN JOHNSON on FIRAXIS engine
https://www.youtube.com/watch?v=VXH9_d_pDa4

SOREN JOHNSON game diary
https://www.designer-notes.com

GAMEBRYO (civ 4) wywiad z:

- John Austin NDL (1998), gamebryo org
- Tim Peterson NDL (1999), mac port of gamebryo; DirectX and PC
- Dan Amerson (2001), gamecube, xbox 360, technical director
- Mike Daly (2004) exporters, toolchain

https://www.gamedeveloper.com/audio/classic-tools-retrospective-the-birth-death-and-re-birth-of-gamebryo-2
^^ sprawdz “recent”

Firaxis' Civilization V: A Case Study in Scalable Game Performance
https://gdcvault.com/play/1012192/Firaxis-Civilization-V-A-Case

SHANON NUMBER (chess)
https://archive.org/details/philosophical-magazine-and-letters_1950-03_41_314/page/256/mode/2up

CIVILIZATION VI turn count
https://www.reddit.com/r/civ/comments/5h2rg6/noob_question_dont_upvote_whats_the_average/

Togelius, Yannakakis
https://www.um.edu.mt/library/oar/bitstream/123456789/29599/1/General_general_game_AI.pdf

TRIBES repo
https://github.com/GAIGResearch/Tribes

OPTIMIZE MEMORY ACCESS PATTERN USING LOOP INTERCHANGE AND CACHE BLOCKING TECHNIQUES
https://www.intel.com/content/www/us/en/docs/advisor/cookbook/2023-0/optimize-memory-access-patterns.html

INTEL SOA
https://www.intel.com/content/www/us/en/developer/articles/technical/memory-layout-transformations.html

MICRO RTS
https://github.com/Farama-Foundation/MicroRTS
and citation:
Santiago Ontañón (2013) The Combinatorial Multi-Armed Bandit Problem and its Application to Real-Time Strategy Games, In AIIDE 2013. pp. 58 - 64.

RYAN GORDON on HoMM 3 Linux port
https://www.icculus.org/~icculus/writing/linux.com/mfc_to_gtk.txt

NOTATKI

Wymagania:

- Wystawienie API dla agenta AI
    - kopiowanie (przenoszenie stanu gry)
    - command visitor?? pattern
    - peek by 1
    - execute
- TANIE kopiowanie stanu gry

Problemy:

- Kopiowanie macierzy NxM jest kosztowne (plansza) [minimax pracuje 1000 godzin, 8 TB RAMu dla głębokości 6 (tak naprawdę 3)]

Pomysł 1:

- Macierz pointerów (duże obj) → problem: kopiowanie NxM pointerów
- Dalej stosujemy minimax/montecarlo
- Zamiast kopiowania GAME_STATE, kopiuje się tylko pointery (shared) oraz stosuje COPY_ON_WRITE, przez co poddrzewa są niezależne, a dzieci nie modyfikują rodzica, bo rodzic jest READ_ONLY. Tj. dziecko ma pointery do obiektów rodzica poza tym jednym, który zmienia, wtedy go kopiuje.

Pomysł 2:

- GPU acceleration???
- +Szybkie operacje
- - 1000 tasków jednowektorowych się nie opłaca (1 task 1000-wektorowy byłby git)

Pomysł 3:

- Statyczna macierz dla nieruchomych obiektów (drzewa, góry, miasta) N x M
- ^ALBO array posortowany leksykograficznie
- ECS dwuwektorowy dla obiektów dynamicznych
- Detekcja kolizji poprzez macierz[x][y], następnie dyn_objs[for each]

Pomysł 3.1:

- pseudo ECS z wektorem pointerów (to samo COPY_ON_WRITE co w p.1)

Pomysł 3.2:

- faktyczny ECS z kopiowaniem całego obiektu, np. HERO (15 statystyk, 7 jednostek, 10 artefaktów, 5 czarów, itp. wszystko na intach albo enumach i koniecznie contiguous w pamięci)
- plus: INSERTION_SORT jedna iteracja, po każdym MOVE
- minus: kopiowanie HERO (ale niby same inty)

Pomysł 4:

- W odpowiedzi na ten problem, projektowany silnik wykorzystuje paradygmat Data-Oriented Design, minimalizując narzut kopiowania poprzez projektowanie stanu gry jako ciągłych bloków pamięci pozbawionych wskaźników, co

umożliwia wykonywanie operacji powielania stanu za pomocą wysoce zoptymalizowanych operacji blokowych."

Notatka:

- Array of Structures (AoS)
    - vs
- Structure of Arrays (SoA) ?= ECS

Notatka:

- C++23: wyrównanie struktury do rozmiaru linii cache (zazwyczaj 64 bajty), co zapobiega *False Sharing* w programowaniu wielowątkowym

```
#include <new>
struct alignas(std::hardware_destructive_interference_size) CacheAlignedData {
    uint64_t counter; // Ta struktura ma gwarancję, że zajmie osobną linię cache
};
```

Notatka:

- Profiler, np. Intel VTune, Perf na Linux, wbudowany w Visual Studio